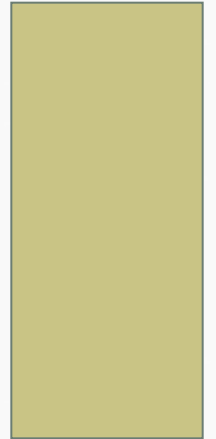


EC-200 DATA STRUCTURES

LECTURE 16



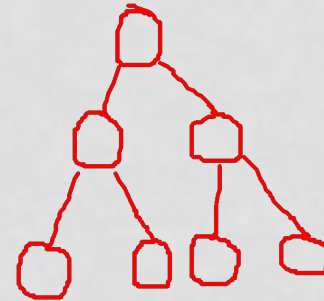
SEQUENTIAL SEARCHES

- how long it will take to find an item matching a key in the collections
 - the worst-case time (Item does not exist)
 - the best possible time (exists in first location)
- If there are ***n*** items in our collection
 - worst case, ***n*** comparisons $O(n)$
 - Best case, 1 comparison $O(1)$

BINARY SEARCH TREE

- Binary search, we first compare the key with root node
- If found, return
- If $<$ root, search left subtree
- If $>$ root, search right subtree

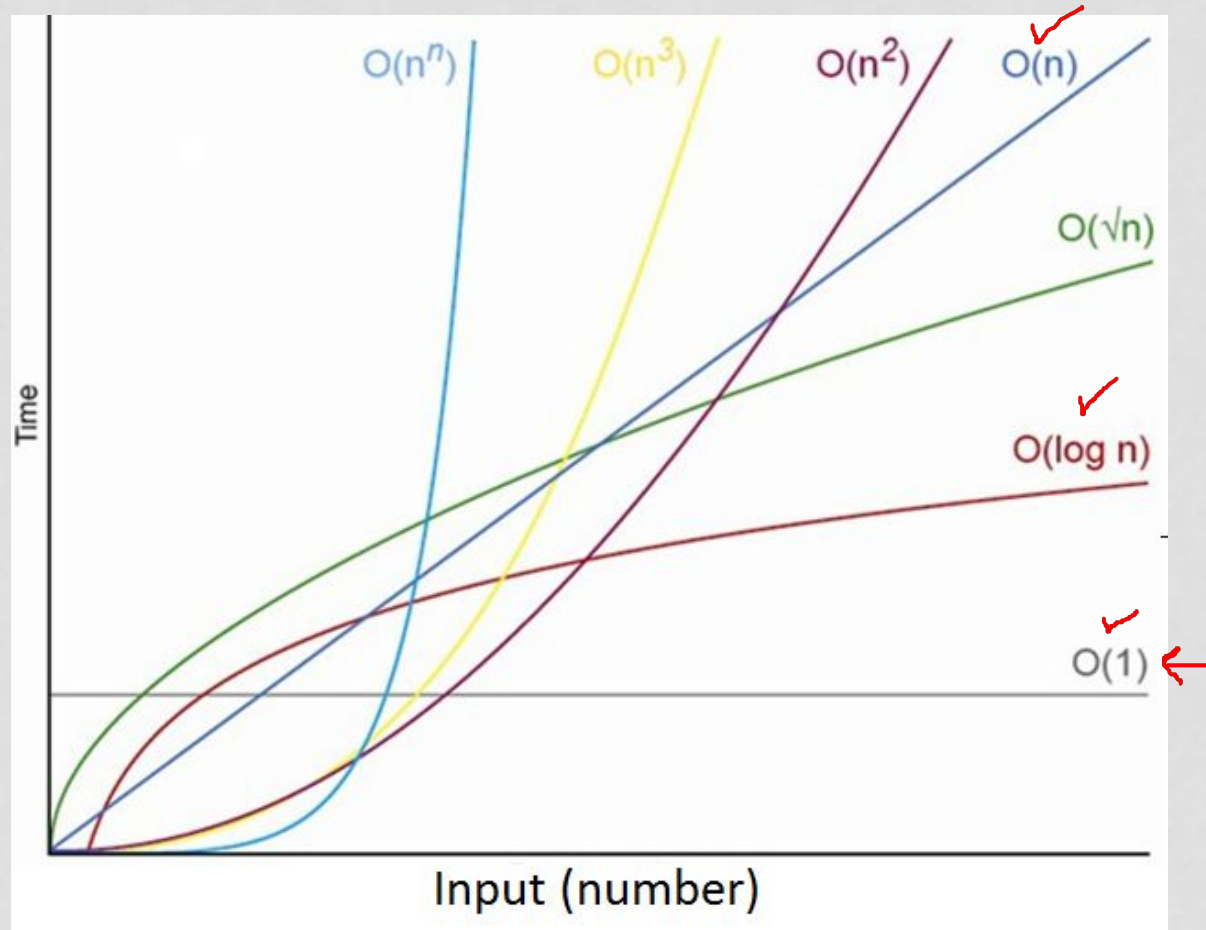
$$\log_2(N)$$



COMPLEXITY ANALYSIS

- Each step of the algorithm divides the block of items being searched in half.
- We can divide a set of n items in half at most $\log_2 n$ times. **HOW ??**
- binary search is proportional to $\log n$ and we say this is a $O(\log n)$

COMPLEXITY ANALYSIS



HASHING

- Sometimes, **time is vital**
 - Call received at **police emergency**: must search for the callers' address fast
 - **Air traffic control** system: must search for flight information corresponding to a flight number fast
- Hashing aims to search for the desired item **instantaneously**
 - i.e. locate the item without searching

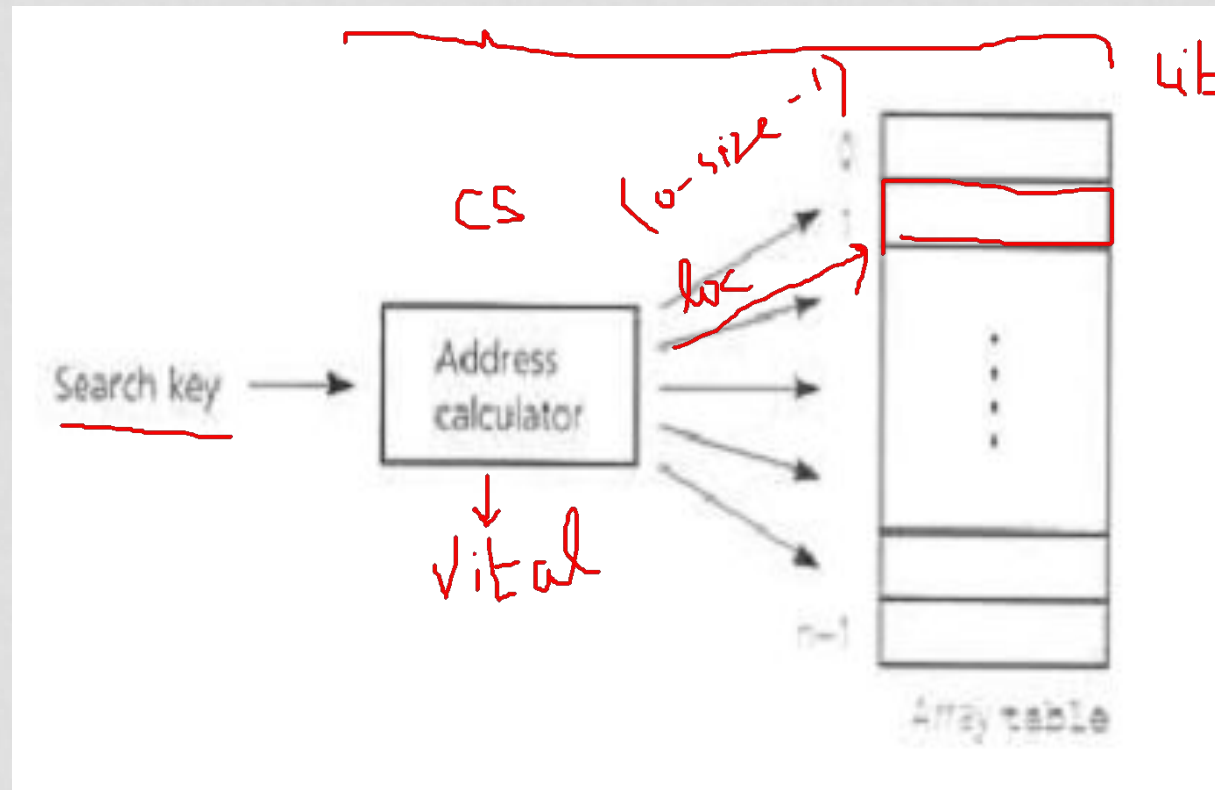
HASHING

$$\propto O(N)$$

- Two ingredients:

- ✓ • An array, called **hash table** or simply table to hold data items
- ✓ • An **address calculator**, that tells the exact location within the hash table where an item should be stored (for insertions) or retrieved

HASHING



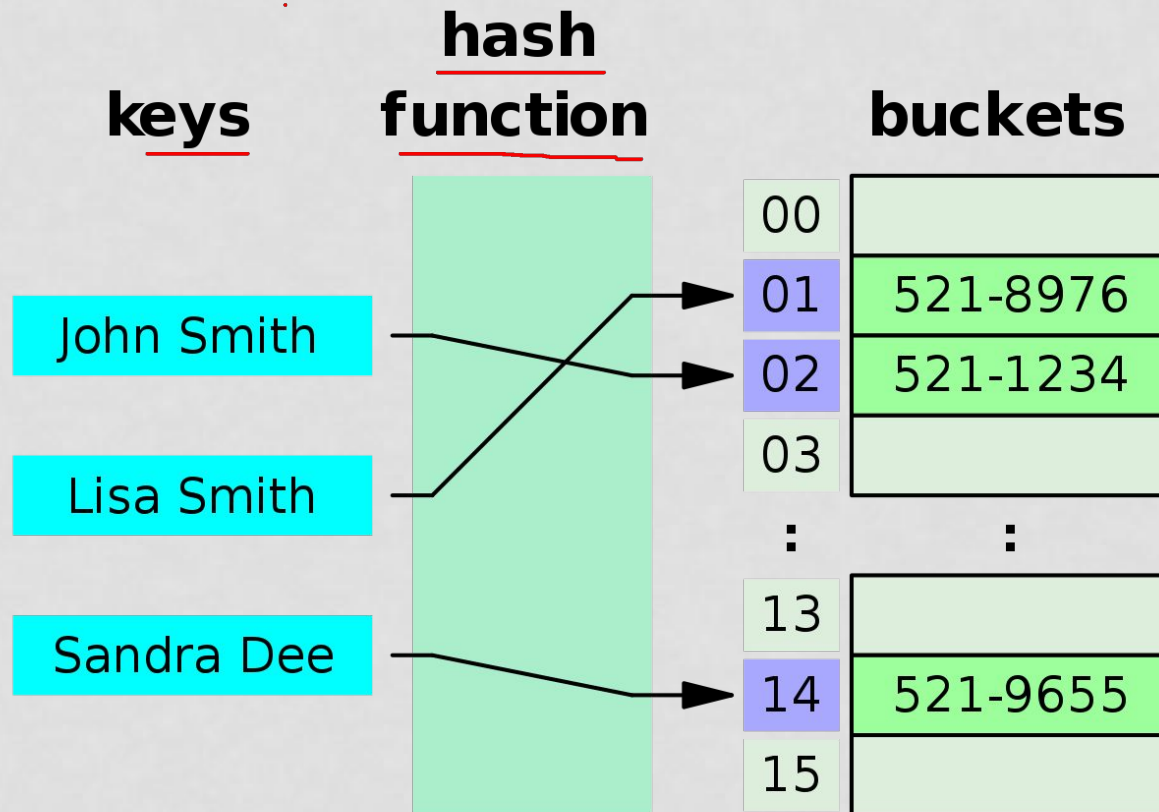
HASHING EXAMPLES

- Some examples of how hashing is used in our lives include:
 - ✓ • In universities, each student is assigned a unique roll number that can be used to retrieve information about them.
 - ✓ • In libraries, each book is assigned a unique number that can be used to determine information about the book, such as its exact position in the library or the users it has been issued to etc.

HASHING



✓ ?



HASH TABLE

$O(1)$

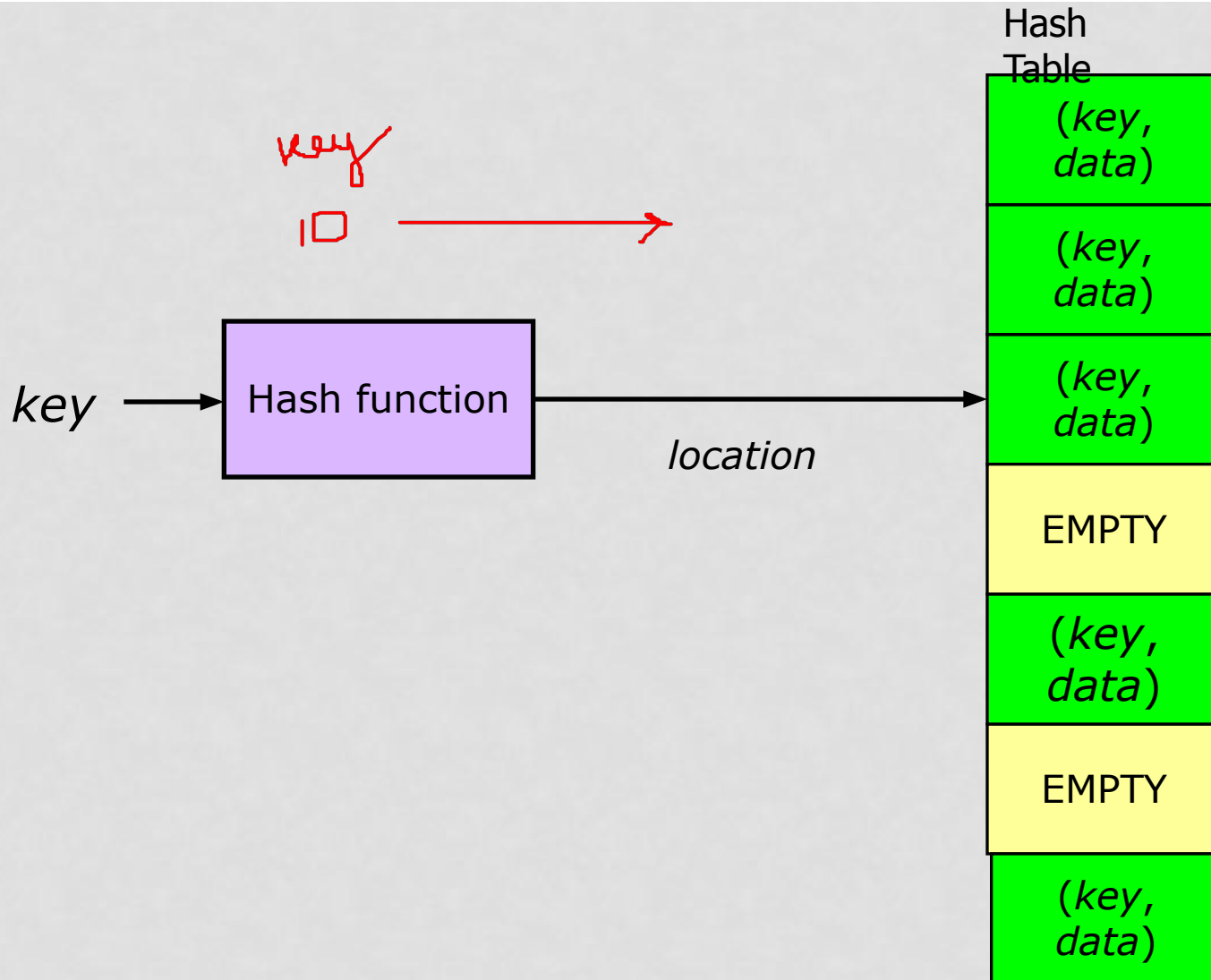
- A **Hash Table** is a data structure in which items are stored according to the location specified by a hash function.
 - Typically the hash table is an array of fixed size
 - Search performed using some part of the data— called the key.
 - Deleting is accomplished by marking a location as “empty”.
 - Thus, **ideally**, search, insert, and delete are all constant-time operations

✓ HASH FUNCTION

- Hash function is an address calculator that maps **keys** to integers (which represent indices into the hash table). Mathematically:

$$\text{Hash}(\text{key}) = \text{Integer}$$

RELATION BETWEEN HASH TABLE AND HASH FUNCTION



INSERTION IN HASH TABLE

  `tableInsert (in newKey, in newData)`

% inserts data identified by newKey into the table

i= table index corresponding to newKey as calculated
by hash function

`table[i].data=newData`

`table[i].key= newKey`

Insertion in Hash Table is $O(1)$

RETRIEVAL IN HASH TABLE

tableRetrieve (in Key)

% retrieves Key and corresponding data from the
%table

i= table index corresponding to Key as calculated by
hash function

if (table[i] contains the Key)

 display corresponding data from table

else

 Key not found

Retrieval in Hash Table is $O(1)$

DELETION IN HASH TABLE

tableDelete (in searchKey)

% deletes searchKey and corresponding data from the
%table

i= table index corresponding to searchKey as
calculated by hash function

if (table[i].key== searchKey)

delete the table entry

else

searchKey not found

Deletion in Hash Table is $O(1)$

IDEAL HASH FUNCTION

- Can be computed quickly. ✓
- Spreads out its results evenly over the possible output values (positions in the table). ?
- Generates unique location for each unique key. ✓

EXAMPLE HASH FUNCTION

- What if we use this hash function

✓ **Hash (Key) == Key**

- What is wrong? 2, 300, 4000, 2
- What if Key varies over a large range?
- Imagine when range of key values is much larger than number of key values
- Unnecessary wastage of space

ANOTHER HASH FUNCTION

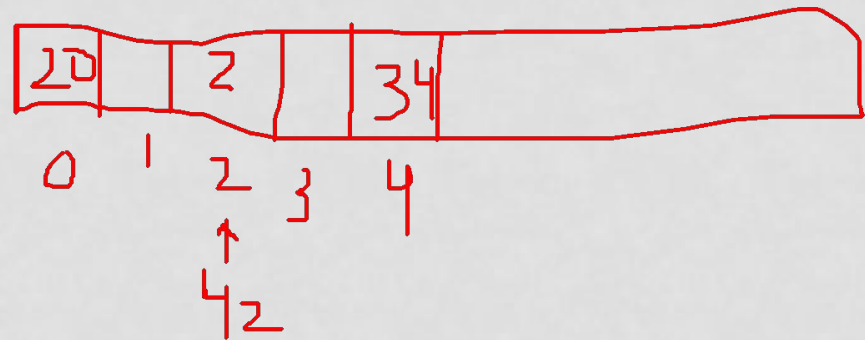
- If $\text{Hash}(\text{Key}) == \underline{\text{Key}} \% \underline{\text{M}}$ (where M is table size)

- What is wrong?

- Suppose

$$M = \underline{10}$$

$$K = \underline{2}, 20, 34, 42, 76$$



- Then $\text{Key} \% M = 2, 0, 4, 2, 6, \dots$

- Since 10 is even, all even Keys are hashed to even numbers ... Greater chance of collisions (particularly for even values of Key)

AN IMPROVEMENT

- If **Hash (Key) = Key % P**, with P a prime number ($\geq M$)
- Suppose
 P = 11 | 
 K = 10, 20, 30, 40
- Key % P = 10, 9, 8, 7
- More uniform distribution...
- Hash table size should ideally be prime

HASH COLLISIONS

The problem with hashing is this:

- Allocating an array large enough for every possible key is inefficient (and often impossible).
- Therefore, we use a small(ish) array (or similar structure).
- With this approach, however, items with different keys may map to the same hash value.
- This is called a **collision**.

HASH COLLISIONS

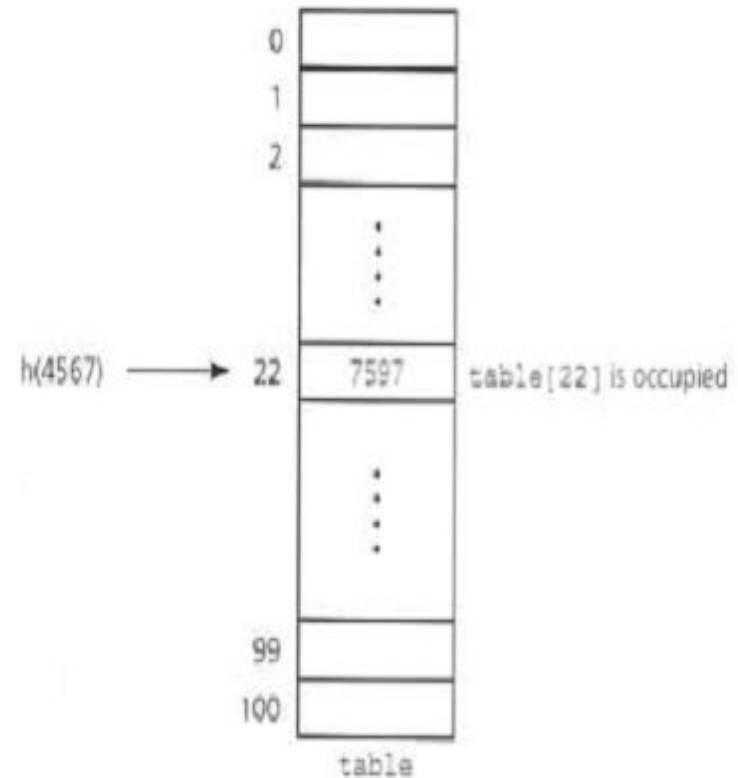
In general:

- Collisions are not a big problem as long as there are not very many of them.
- But there can conceivably be lots of them, so it is necessary to be prepared for collisions.

HASH COLLISIONS

Example

- Hash Key=7597 and Key=4567 in a table of size 101 using the hash function $\text{hash}(\text{Key}) = \text{Key} \% \text{table size}$
- $\text{Hash}(7597) \% 101 = 22$
- $\text{Hash}(4567) \% 101 = 22$



COLLISION RESOLUTION

1. Open Addressing ✓

- Linear Probing
- Quadratic Probing
- Double Hashing
- Re-Hashing

2. Restructuring the table ✓

- Chaining
- Bucketing

COLLISION RESOLUTION: APPROACH 1

Open Addressing

- The Hash Table is essentially an array where each location can store a **single record (i.e. key and data)**.
- If we get a collision, we **probe** (look for) another available location within the table
- The sequence of locations that are examined is called a **probe sequence**

LINEAR PROBING

- We look at location $hash(key)$, then $hash(key)+1$, then $hash(key)+2$, etc.
- Wrap around from last location to first if necessary
- Linear probing tends to form **clusters**, which slow things down.



PSEUDOCODE: INSERTION WITH LINEAR PROBING

`Insert(key) // assume unique keys`

1. `index = key % table_size;`

2. `if (table[index] == 'EMPTY')`
`table[index] = key;`

3. `Else {`
`index++;`
`index = index % table_size;`
`goto 2;`
`}`

PSEUDOCODE: SEARCH WITH LINEAR PROBING

Search (key)

1. Index = key % table_size;
2. If (table[index] == 'EMPTY')
 return -1; // key not found
3. Else if (table[index] == key) return
 index;
4. Else {
 Index ++;
 index = index % table_size;
 goto 2;
 }

LINEAR PROBING EXAMPLE

Insert 89, 18, 49, 58, 69 using Hash(Key)=Key%10 with linear probe

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

(Notice the clusters)

QUADRATIC PROBING

- Probe sequence $hash(key)$, $hash(key)+1^2$, $hash(key)+2^2$, $hash(key)+3^2$, etc.
- Avoids clusters

QUADRATIC PROBING EXAMPLE

Insert 89, 18, 49, 58, 69 with $\text{Hash}(\text{Key}) = \text{Key} \% 10$ and quadratic probe

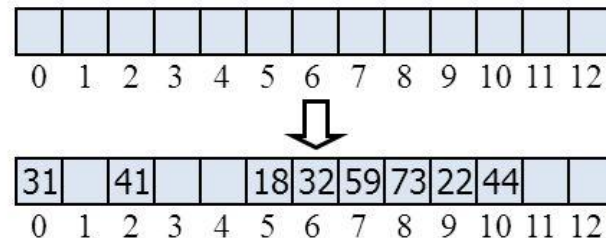
	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

DOUBLE HASHING

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing
 - $N = 13$
 - $h(k) = k \bmod 13$
 - $h'(k) = 7 - (k \bmod 7)$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$h'(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	



Collision handling:

$A[(i + f(j)) \bmod N]$ where $i = h(k)$
for $j = 1, 2, 3, \dots$, where $f(j) = j * h'(k)$

$h'(k) = q - (k \bmod q)$ where $q < N$ and q is a prime

REHASHING

- When using open addressing, hash table may get *too* full
 - Insertions, deletions, search take longer time
- **Solution: Rehash**
 - **Build another table** that is twice as big and has an **adjusted hash function**
 - **Move all elements** from smaller table to bigger table using the adjusted hash function
- **Cost of Rehashing = $O(N)$**
 - But happens only when table is close to full
 - Close to full = table is X percent full, where X is a tunable parameter

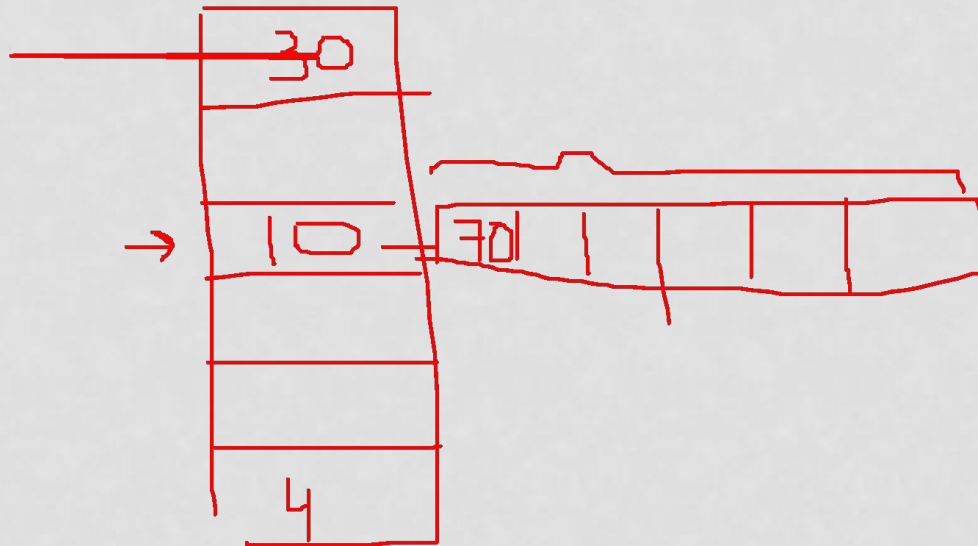
70%

COLLISION RESOLUTION: APPROACH 2

- Restructure the hash table
 - Structure the table such that each location can accommodate more than one item.
 - Two possible ways to structure the table
 - ✓ Buckets
 - ✓ Separate chaining

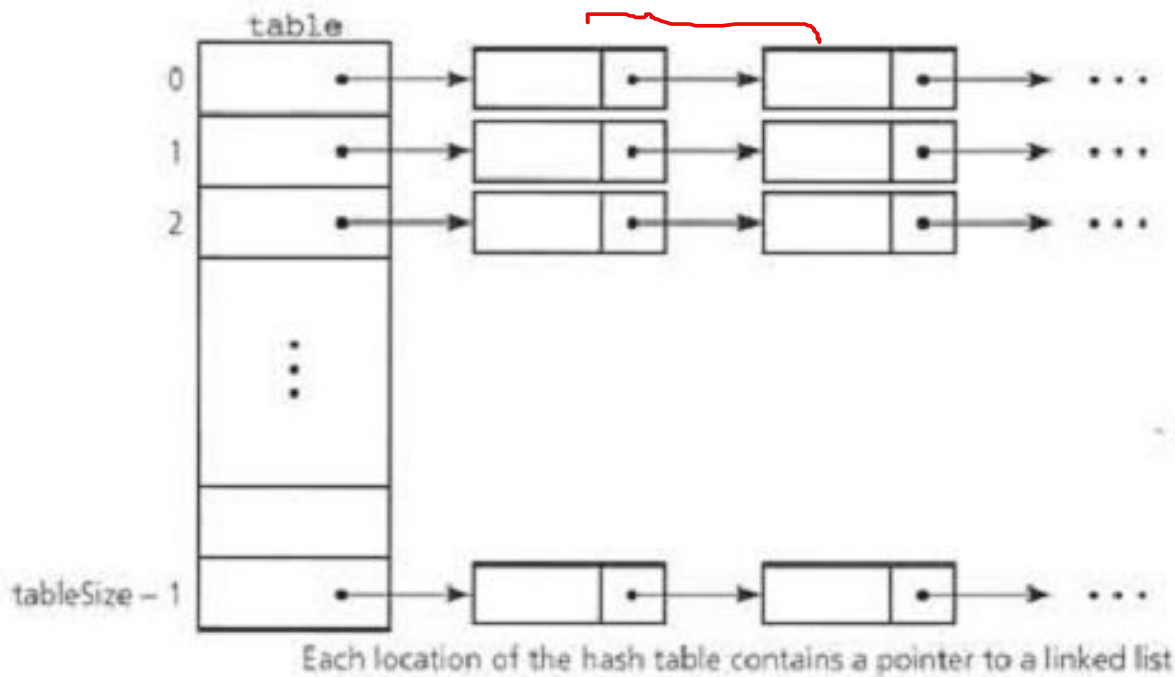
BUCKETS

- Each location $\text{table}[i]$ within the table is itself an array
- Deciding the size of buckets is a problem



SEPARATE CHAINING

- Every location $\text{table}[i]$ in the table is a linked list

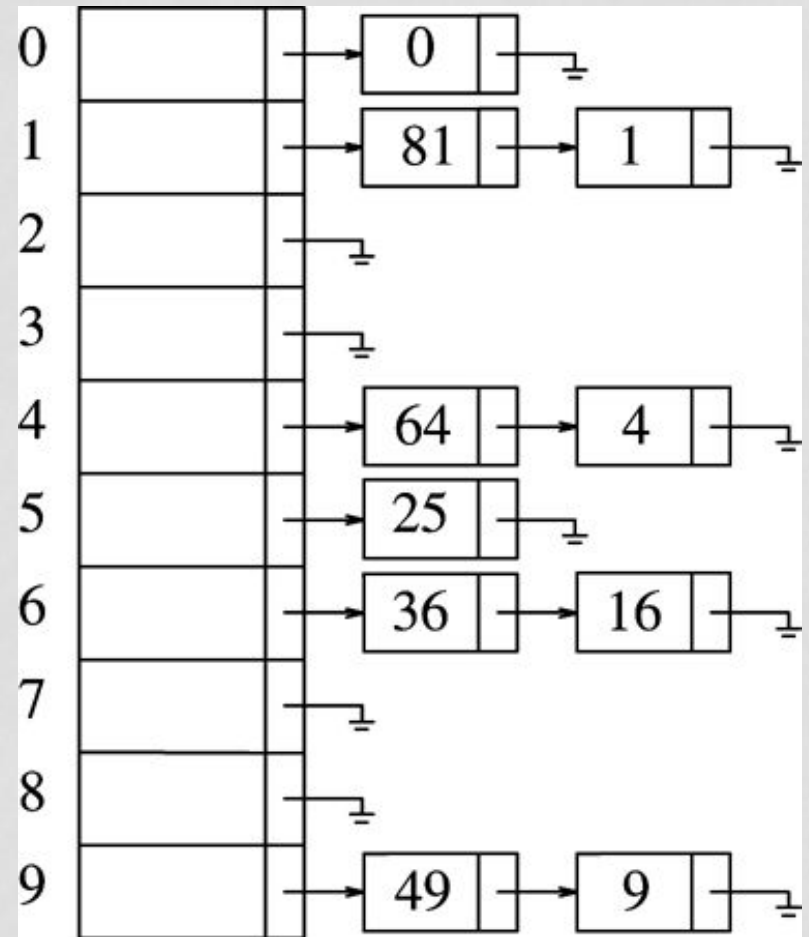


SEPARATE CHAINING EXAMPLE

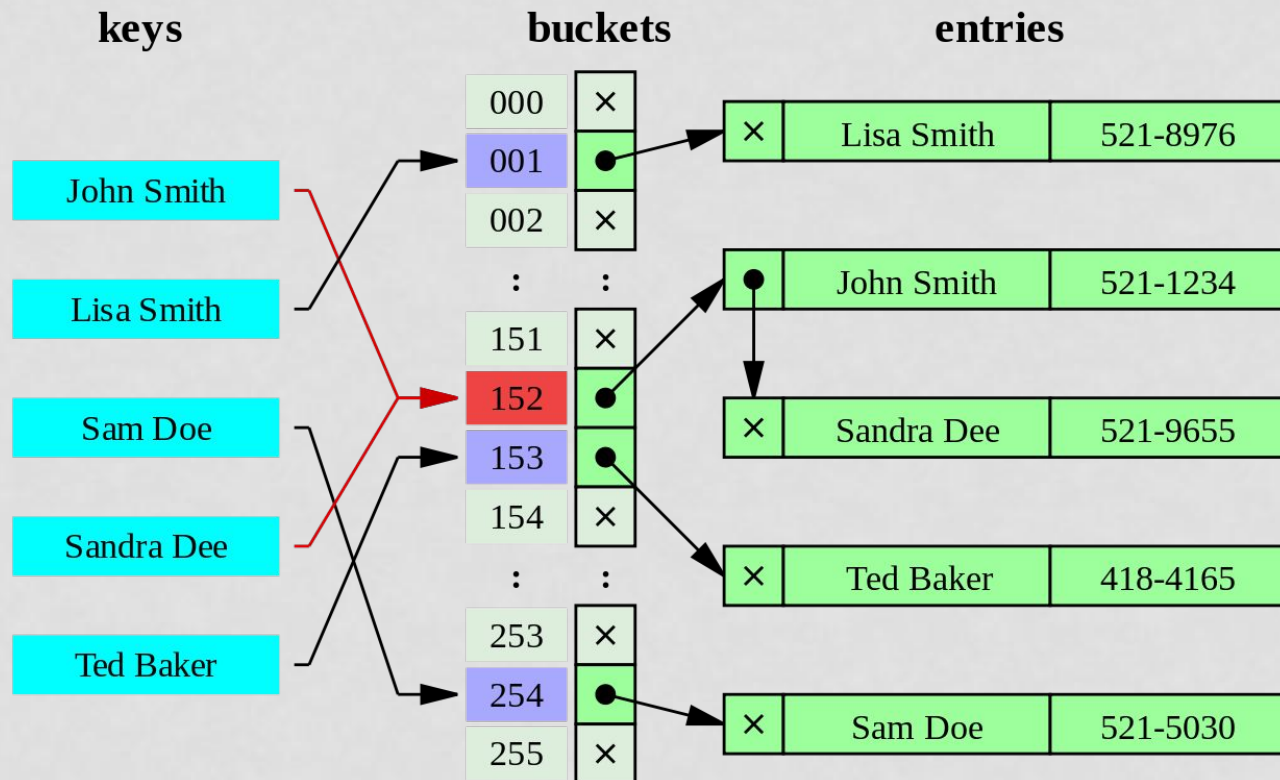
- Each table entry stores a list of items
- So we don't need to worry about multiple keys getting mapped to the same entry.

• **Example: hash the keys**

49, 0, 9, 64, 1, 4, 81, 25, 16, 36 into a table of size 10



CHAINING



READING

- Chapter 12, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.

HELP

- <https://www.geeksforgeeks.org/applications-of-hashing/>